

AIPL 型システムの健全性境界レポート

Phase 11–15 を経た現時点 (2026-05-10) のスナップショット

Yasushi Kodama (kodamay@)

2026-05-10

Abstract

本レポートは、アクター指向並列言語 **AIPL** (Actor-based Intelligent Parallel Language) の型システムが Phase 11–15 を経て現在カバーしている領域と、そうでない領域を整理する。AIPL の型システムは段階的型付け (gradual typing) を基礎に、Capability 効果系 (Phase 12)、CSP チャンネル (Phase 13)、線形/借用型 (Phase 14)、アクターフィールドのカプセル化 (symbol_owned, Phase 15) を順に積み上げてきた。それぞれの軸について「何を静的に保証できるか」、「どこで健全性が崩れるか」、「現在ランタイム監視がどこまで補完しているか」を述べる。結論として、AIPL は注釈された宣言領域については局所健全であり、未注釈領域および動的反射機能との境界では健全性が成り立たないという、gradual な処理系として典型的な特性を持つ。

Contents

1 はじめに	1
2 型システムの全体像	1
2.1 Phase ごとの型システム拡張	1
2.2 設計原理: gradual + capability + linear + owned	1
3 型推論できる部分 (健全性が成り立つ領域)	2
3.1 宣言済み関数・メソッドの呼出サイト	2
3.2 ビルトイン演算と typeof narrowing	2
3.3 構造的レコード型・タプル型・固定長配列	2
3.4 暗黙ジェネリクス (Phase 11c)	3
3.5 Capability 効果系 (Phase 12)	3
3.6 線形/借用 (Phase 14)	3
3.7 symbol_owned: アクターフィールドのカプセル化 (Phase 15)	3
4 型推論できない部分 (健全性が崩れる領域)	4
4.1 未注釈領域 (gradual の代償)	4
4.2 動的コンパイル・メソッド注入・become	4
4.3 動的メッセージ送信 (send target.method(...))	4
4.4 高階関数・クロージャ未対応	5
4.5 再帰関数の効果推論	5
4.6 配列添字の動的境界	5
4.7 チャンネル要素型の境界越え	5
4.8 レコードに格納されたアクター参照	5
4.9 typeof narrowing の限界	6
4.10 AI 呼び出し・ファイル/画像 I/O の戻り値型	6
4.11 Linear と動的ディスパッチの相互作用	6
5 ランタイム監視による補完	6

6	まとめと今後	7
6.1	健全性のステートメント (現状)	7
6.2	次に手を入れるべき優先順位	7
6.3	結論	7

1 はじめに

AIPL は OCaml と Python の二言語処理系を持つアクター指向並列言語である. 本レポートが対象とするのは Python 実装に同梱された静的型検査器 `aipl_typeck.py` および対応するパーサ・AST・インタプリタである.

健全性 (soundness) を本レポートでは以下の意味で用いる:

Progress 型付け済みのプログラムの実行は, 型に起因する型不整合 (例: 文字列を数値演算する) で stuck しない.

Preservation 評価ステップを進めても, 値の型は宣言された型と互換である (`any` を含む単純な部分順序の下で).

ただし AIPL は段階的型付けであるから, 「型付け済み」とは プログラム中の注釈された部分を指す. 未注釈の部分は `any` として扱われ, 検査器は意図的に黙認する. この境界が健全性を緩める方向に作用するため, 本レポートでは「どこで黙認しているか」を列挙することに重点を置く.

2 型システムの全体像

2.1 Phase ごとの型システム拡張

Phase	軸	追加された静的検査
11a	関数注釈	パラメータ・戻り値の型注釈と呼び出しサイト整合
11b	共用体型	<code>T1 T2</code> と <code>typeof</code> narrowing
11c	ジェネリクス	単一文字型変数の暗黙多相 (<code>T U V...</code>)
11d	構造的レコード	<code>record{a:int, b:string}</code> の field type 推論
11e	長さ付き配列	<code>array[T, N]</code> で要素アクセスの境界条件
12	Capability 効果	<code>!{fs, ai, net, mut}</code> の宣言と呼出連鎖伝播
13	CSP チャネル	<code>channel[T, cap=N]</code> 要素型の <code>send/recv</code> 整合
14	線形/借用	<code>linear T</code> の use-after-move 検出
15	<code>symbol_owned</code>	アクターフィールドの可視性 (<code>pub</code>) と外部書込禁止

Table 1: Phase 11–15 で追加された静的検査の概要

2.2 設計原理: gradual + capability + linear + owned

AIPL の型システムは次の四つの 独立した軸を組み合わせる:

1. **Type axis:** 値が何の型か (`int` / `string` / `record{...}` など)
2. **Effect axis:** 関数が何の副作用を起こすか (`fs`, `net`, `ai`, `mut`)
3. **Linearity axis:** 値が何回使えるか (`linear T` なら高々 1 回)
4. **Ownership axis:** 状態を誰が変更できるか (アクターのフィールドはそのアクターのメソッドからのみ書込可)

これらは互いに直交であり, 検査器は同じ AST 走査の中で四つを並行に追跡する. `linear int` のような値は型 `int` かつ線形で, かつ `!{mut}` 効果を持つ関数のみが消費するといった組合せが起きうる.

3 型推論できる部分 (健全性が成り立つ領域)

3.1 宣言済み関数・メソッドの呼出サイト

注釈付きの関数は呼び出し時に厳格にチェックされる.

```
function add(a: int, b: int) -> int { return a + b; }

var x: int = add(3, 4);           // OK
var y: int = add("3", 4);        // □ 型不整合
var z: string = add(3, 4);       // □ 戻り値型不整合
```

保証: 関数本体・引数・戻り値の型が注釈と整合することを, 呼出サイトで検査器が確認する.any を経由しない限り Preservation が成り立つ.

3.2 ビルトイン演算と typeof narrowing

+, -, *, /, == などのビルトイン演算はオペランド型を推論する.int + int = int, string + any = string (連結) など, オーバーロード規則が aipl_typeck に内蔵されている.

typeof は値の動的型を文字列で返す. 以下のような形は静的に narrowing される:

```
var v: int | string = pick();
if (typeof(v) == "int") {
  var d: int = v + 1;    // OK: v は int に絞り込まれる
} else {
  var s: string = v;     // OK: v は string に絞り込まれる
}
```

3.3 構造的レコード型・タプル型・固定長配列

record{a:int, b:string} のリテラルおよびフィールドアクセスは静的に型付けされる.

```
var r = { name: "Alice", age: 30 }; // 推論: record{name:string, age:int}
var n: string = r.name;             // OK
var a: int = r.age;                 // OK
var b: bool = r.name;               // □
```

長さ付き配列 array[T, N] については, 要素アクセス a[i] の i がリテラル整数で $0 \leq i < N$ に収まるかを検査器が確認する (動的添字は静的不能なので any となり Preservation の保証から外れる).

3.4 暗黙ジェネリクス (Phase 11c)

array_get(xs: array[T], i: int) -> T のような注釈を, 検査器は単一文字型変数 T を type variable として unify する. 呼出サイトで xs: array[int] なら T = int に確定し, 戻り値型は int. Higher-rank polymorphism は無し (引数として関数を受け取る関数は別途 any 扱い).

3.5 Capability 効果系 (Phase 12)

関数宣言の !{fs, net} のような注釈は, 呼出連鎖を通じて伝播する.

```
function read_url(u: string) -> string !{net, fs} { ... }
function load_cfg() -> string !{fs} {                // □ net が欠けている
  return read_url("...");
}
```

保証: 注釈付きの関数の宣言効果集合は, その関数本体が呼び出す注釈付き関数全ての効果集合を上集合として含む.

境界: 未注釈関数 (effects=None) は伝播対象外. 動的に合成される関数 (Section 4) も対象外.

3.6 線形/借用 (Phase 14)

linear T を取る関数に変数を渡すと、その変数は *moved* とマークされ、以降の参照は use-after-move エラー:

```
var fh: linear int = open("x");
var n = write(fh, "hello"); // fh consumed
close(fh);                  // □ use-after-move
```

制御フロー認識: if の then 枝が return で終了する場合、その枝で moved となった変数は post-if 時点では生きていと扱われる (各 return 経路で 1 回だけ消費されれば OK).

境界:

- while ループ内での消費は、各反復で重複消費とならないようループ全体を 1 回の摩耗とみなす保守的扱い.
- クロージャ・高階関数は未実装 (引数として linear 値を渡す関数を間接的に渡す経路は無い).
- send obj.method(linear_val) のような非同期送信における所有権移譲は 宣言上は *consume* だが、送信先が消費しない場合の検出は静的には不能 (受信側のメソッドの線形パラメータ宣言を参照することで部分的に検査される).

3.7 symbol_owned: アクターフィールドのカプセル化 (Phase 15)

アクターのフィールドは既定で外部不可視 (private).pub 修飾子を付けたものだけが外部から read 可能となり、外部からの書込みは pub・private 問わず常に禁止される.

```
class Account {
  var balance: int = 0;
  pub var holder: string = "";
  method deposit(n: int) -> int { balance = balance + n; return balance; }
}
class Bandit {
  method run() {
    var a = new Account();
    var s = a.holder; // OK
    var b = a.balance; // □ private read
    a.balance = 999; // □ external write (pub でも禁止)
  }
}
```

保証: アクターの不変条件 (例: balance >= 0) を破壊するにはそのアクターのメソッドを通すしかなく、メソッド内で行う検査・補正がステート整合性を担保できる.

4 型推論できない部分 (健全性が崩れる領域)

ここからは 健全性が静的には成り立たない領域を列挙する. gradual typing の代償として黙認しているもの、動的機能との境界で原理的に不能なもの、そして未実装のものに分類する.

4.1 未注釈領域 (gradual の代償)

注釈の無い変数・パラメータ・戻り値は内部的に any 型を持ち、任意の操作・任意の型への代入を黙認する.

```
function f(x) { return x + 1; } // x は any
var s: string = f("abc");      // OK 扱い (型整合の証明はない)
var n: int = f("abc");          // OK 扱い (実行時に型エラー)
```

原因: gradual typing の中核仕様. **現状の補完:** 無し (ランタイムキャスト未実装). **将来案:** any 境界で runtime check を挿入する Reticulated Python 流の transient soundness.

4.2 動的コンパイル・メソッド注入・become

```
var src = "class Greeter { method hi(n) { print(\"hi \" + n); } }";
var g = compile(src); // 戻り値: 動的に作られたクラス参照
var obj = new_instance(g);
send obj.hi("Alice"); // メソッド hi の signature は不明
```

`compile()`, `add_method()`, `remove_method()`, `become C()` はいずれも実行時にクラス形状を変更する. 静的検査器はこれらの呼び出し以降のオブジェクトの型を `any` に退化させる (または `actor(?)` のような `unknown` を保持する).

原因: 動的反射 (reflective metaprogramming) と静的型付けは原理的に相容れない. 現状の補完: 無し.

4.3 動的メッセージ送信 (`send target.method(...)`)

`send` は `actor` 変数の動的解決によって行われる:

```
class Demo {
  var partner = 0; // 後で set される actor 参照
  method run(other) {
    partner = other; // partner: any (actor 型の宣言がない)
    send partner.tick(); // tick の signature は静的不明
  }
}
```

`partner` に注釈が無い限り受信側のクラスは決まらない. `now` 呼び出しの場合は戻り値型も `any`.

現状の補完: クラス内で `now self.method(args)` のように `self` に対する呼出は静的にメソッド辞書を引いて検査する. `var p: Counter = ...; now p.method(args)` のように `actor(C)` 型注釈があれば呼び出しサイトで `checking` が働く.

境界例:

```
var x: actor(Counter) = new Counter();
become OtherClass(); // x の実体クラスが入れ替わる可能性
send x.tick(); // 静的型は actor(Counter) のまま
```

`become` は静的型注釈と実体型の乖離を生む典型例である.

4.4 高階関数・クロージャ未対応

AIPL は関数値 (`FunctionRef`) は持つが, これを引数にする注釈構文は未提供である. 以下のような関数経由のコールバックは検査器を素通りする:

```
function each(xs, f) { ... } // 未注釈
each([1,2,3], print); // f の signature 検査なし
```

原因: 未実装. 将来案: `(int) -> int !{net}` のような関数型構文を導入し, 引数の型・効果集合を含めて `unify` する.

4.5 再帰関数の効果推論

宣言された効果を持つ関数 `f` が `f` 自身を呼ぶ場合, 検査器は単純な不動点を取らない. 実装は「自分自身の効果を読みに行く」ため宣言効果に閉じる方向にバイアスされており, 未注釈の再帰呼出経路で間接的に発生する効果は検出されない.

4.6 配列添字の動的境界

固定長配列 `array[T, 3]` は `a[2]` までを許可するが, `a[i]` で `i: int` が動的値の場合は静的に範囲を確認できない.

```
var a: array[int, 3] = [10, 20, 30];
var i = read_int();           // any
var v = a[i];                 // 境界外可能 (静的不能)
```

現状の補完: ランタイムは Python リストの境界例外でクラッシュ. 将来案: 依存型 ($i < N$ の証明) または refinement の導入.

4.7 チャンネル要素型の境界越え

channel(0, "int") のような注釈チャンネルは, その変数の型として channel[int] を保持する. しかしこの変数を未注釈関数の引数に渡すと型情報が any に退化する:

```
var ch = channel(0, "int");
function relay(c) { channel_send(c, "abc"); } // 注釈なし
relay(ch);                                // □ 検出されない
```

現状の補完: ランタイムは type assertion で送信時に要素型をチェックする (_AiplChannel.send 内部). 静的 + 動的の混合保証.

4.8 レコードに格納されたアクター参照

symbol_owned (Phase 15) は var a = new C(); a.field = ... の形だけを取り締まる. 以下のような迂回は検査器の対象外:

```
class C { var k: int = 0; }
class D {
  method bad() {
    var c = new C();
    var box = { actor: c };
    box.actor.k = 99;           // □ レコード経由の write
  }
}
```

実際にはランタイム側でも record[attr] は dict mutation として通ってしまうため, ここは静的にも動的にも防御がない.

将来案: FieldAssign の base がレコードの場合に再帰的にアクター含有を検出する.

4.9 typeof narrowing の限界

narrowing は if (typeof(v) == "T") の形でのみ動作する. 複合条件・ネスト構造・ループ内変動は対応しない:

```
var v: int | string = pick();
if (typeof(v) == "int" && v > 0) { // && 越しの narrowing なし
  var d: int = v;                  // □ v は int|string のまま
}
```

将来案: 条件式の構造解析を depth 制限付きで導入.

4.10 AI 呼び出し・ファイル/画像 I/O の戻り値型

ai_call("...") は実行時に LLM プロバイダから受け取る値であり, 戻り値型は any.file_read, image_open 等も同様で, 本質的に any-typed の境界となる. この境界をまたぐ値は以降 gradual に類する黙認領域に入る.

現状の補完: ビルトイン署名で部分的に string 等を固定しているもの (file_read_text: (string) -> string) はある.

4.11 Linear と動的ディスパッチの相互作用

線形値を非同期メッセージで送る場合, 受信側のクラスが動的決定であれば消費の有無を静的に判定できない:

```
var fh: linear int = open("x");
send some_actor.handle(fh);    // some_actor の handle 受け取りは linear?
```

some_actor: actor(C) の注釈があり, C.handle の宣言が linear パラメータを取ると注釈されていれば検査される. そうでなければ fh の moved 状態は決定できず保守的に moved とマークされる (false negative ではなく false positive 寄り).

5 ランタイム監視による補完

現状 AIPL は ランタイム型キャストを挿入していない. すなわち gradual typing の中核要件である「any 境界での dynamic check」を持たない. ランタイムが補完しているのは以下に限定される:

- 算術演算: Python の演算子例外で型不整合がクラッシュする.
- 配列境界: Python リストの IndexError が伝搬する.
- チャネル要素型: _AiplChannel.send で declared element type に対する isinstance 確認.
- メソッド未定義: 動的ディスパッチで KeyError を投げ, メッセージは drop される (Actor が再開可能).
- レコードフィールド未定義: None に退化 (例外なし).

これは **transient soundness** にも届かない弱い保証であり, 未注釈領域から注釈領域への型違反値の流入を確実に捕えられない.

6 まとめと今後

6.1 健全性のステートメント (現状)

非形式的に述べる. **ローカル健全性 (local soundness)**: 注釈付きの関数 f の本体において, 各局所変数の型は宣言と整合し, 呼出サイトで渡される実引数も注釈と整合することを確認器は確認する.

大域健全性 (global soundness) は成り立たない. 以下の場合に注釈の意味から逸脱する値が観測されうる:

- 未注釈領域から注釈領域へ any 値が流入する.
- 動的反射 (compile / become / add_method) でクラスの形状が変化する.
- 動的にディスパッチされる送信先の signature が宣言と乖離する.
- レコード経由でアクターのフィールドが書き換えられる.

6.2 次に手を入れるべき優先順位

1. **transient cast** の挿入: 注釈境界で runtime check. 最小コストで観測可能な不健全を捕える. (2026-05-10 Phase 16 で実装済み: --transient フラグで有効化, 3つの境界 (var-decl / メソッド引数 / 関数引数) で静的検査の _compatible と同一規則でランタイム型チェック.)
2. **関数型構文**: $(T) \rightarrow U !\{\text{eff}\}$ を導入し高階関数を検査対象に組み込む.
3. **record** 中のアクター参照を symbol_owned で守る.
4. **narrowing** の合成条件対応 (&& 連鎖).
5. **become** に対するクラス変更追跡 (actor(C) 型を become 後に actor(C') に更新).

6.3 結論

AIPL は Phase 15 をもって Phase 9 の言語進化シミュレーションが予測した 4 軸 (type / effect / concurrency / ownership) を全て実装した. 個々の軸は局所健全に動くが, 軸を横断する場面 (動的ディスパッチ越し, 未注釈境界越し, 反射機能経由) では保証が落ちる. gradual typing 言語としては典型的な状況であり, 次の段階は静的保証を諦める方向ではなく, **境界における runtime cast 挿入と 反射機能と型情報の共存設計**に向けて進むのが望ましい.